Exercicios do tema e o seu valor:
1-representación dunha serie de datos (1p), 2-representación e axuste (1.5p), 3-representación de varias series de datos (1.5p), 4-representación, axuste e delimitación de puntos críticos (2p).

Análise de datos

Imos realizar unha práctica para estende-los coñecementos adquiridos en Python e aplicalos á análise e tratamento de datos. Esta práctica divídese en tres subapartados: *importación, representación, axuste e exportación.* Traballaremos cos datos declarados á *Red Nacional de Vigilancia Epidemiológica (RENAVE)* sobre a COVID-19 e que se atopan localizados na seguinte páxina: `https://cnecovid.isciii.es/covid19/#documentación-y-datos`, no ficheiro*casos_tecnica_ccaa.csv* ou en *casos_hosp_uci_def_sexo_edad_provres.csv*, onde se atopan datos máis desglosados.

Importación de datos

Nos numerosos campos da investigación, a principal materia coa que se traballa son **os datos**. Existen multitude de formas de almacenalos para logo analizalos. Cando estes son recollidos no ordenador, soen utilizarse diferentes formatos, como os *.txt, .dat, .xls/.xlsx, .csv, .ods...*

Dentro das *librerías* de *Python* máis empregadas existen xa funcións para importar (cargar en Python) unha ampla variedade de tipos de ficheiros. Amósanse as máis comúns, e centrarémonos na *librería Pandas*, deseñada especificamente para a xestión de datos.

- **Importación manual**
  Coa mesma linguaxe de *Python*, sen utilizar ningunha *librería*, podemos abrir case calquera ficheiro co comando *open()*. Esta función pode ser empregada tanto para ler ficheiros como para escribilos, dependendo da modalidade que se use:

  - *open*(direccion_ficheiro,modo), sendo os modos:
    r: Abre o ficheiro para ser lido.
    r+: Abre o ficheiro para ler e escribir.
    w: Abre o ficheiro para ser escrito.
    w+: Abre o ficheiro para escribir e ler.
    a: Abre o ficheiro para engadirlle elementos (ó contrario que en escritura, que borra o ficheiro previamente!).
    a+: Abre o ficheiro para engadirlle elementos e ser lido.

    open() crea un obxecto do tipo *file*, que ten diferentes propiedades, do mesmo xeito que un *numpy.array* ten unhas funcións particulares. Neste caso, se o o ficheiro foi aberto para lectura, podemos extraer del a información a través de diferentes funcións:

    ficheiro.read(): le o ficheiro e transforma todo o seu contido nunha cadea de caracteres (*string*). Esta función debería ir da man de *split("elemento")*, unha función vinculada ós *strings* que os separa por *"elemento"* e engade cada parte da separación nunha lista (palabra="ab;cd;ef", palabra.split(";")->["ab","cd","ef"])
    ficheiro.readlines(): le o ficheiro liña por liña, almacenando cada unha delas como unha palabra (cadea de caracteres ou *string*) dentro dunha lista. Moi útil para logo iterar sobre o contido.

- **Importación con *numpy***
  Da librería *numpy*, pode empregarse a función *loadtxt* que nos carga en *Python* os datos gardados nun ficheiro. A función intentará transforma-los datos almacenados no ficheiro nun mesmo tipo,

polo que non resulta moi útil para documentos con diferentes tipos de datos (palabras, días, números...). Porén, pode ser extremadamente útil para os datos máis usuais do laboratorio, onde se gardan as os valores de certas medidas en columnas, coa única salvedade de indicarlle que non lea a fila correspondente ó nome das columnas.

- *numpy.loadtxt*(direccion_ficheiro,*args), con múltiples argumentos, entre os que nos poderían ser útiles:
  dtype: o tipo de datos que pretendemos que lea. Por defecto é un *float* (número decimal)
  delimiter: se o ficheiro ten varias columnas separadas por algún elemento (unha tabulación, unha coma, un punto e coma..) indicaríamoslle aquí como identificalas
  skiprows: un número enteiro que indique cantas filas ten que saltarse
  usecols: un número enteiro ou unha secuencia de enteiros (tupla, lista ou *array*) que indique cales son as columnas que ten que recoller, no caso de que non as queiramos todas

- **Importación con *Pandas***
  Pandas (de Panel Data) é unha das librerías en *Python* deseñadas para a manipulación e tratamento de datos e está baseada en *numpy*, polo que poden ser empregadas funcións desta última. A familiarización con esta librería pode resultarvos útil a curto e medio prazo, tendo moita información dispoñible na súa propia páxina: `https://pandas.pydata.org/` e aquí nunha breve introdución de 10 min `https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html`. Con *Pandas* poden crearse varios tipos de obxectos nos que garda-los datos, mais só falaremos dos *DataFrame* que, coa súa estrutura de filas e columnas, son os máis semellantes ás follas de cálculo.

  Un *DataFrame* é un tipo de estrutura de datos onde a información se atopa recollida en forma de matriz e onde tanto as súas filas como as súas columnas poden ser accedidas tanto por índices como polas súas etiquetas/nomes (*labels*). Esta clase ten as seguintes propiedades:

  - *pandas.DataFrame*
    data: os propios datos almacenados
    index: índice ou nome asociado ás filas
    columns: índice ou nome asociado ás columnas
    dtype: tipo de variable vinculado a cada columna

  A importancia deste tipo de estrutura de datos é que calquera das funcións de *Pandas* de importación e exportación a vai empregar. E hai **moitos** tipos de ficheiro que *Pandas* interpreta correctamente, incluso identificando o tipo de variable máis axeitada. Dos tipos máis comúns ou que vos poder servir *xa*, son:

  - *pandas.read_excel*(direccion_ficheiro,*args): para abri-los ficheiros de *Excel* e importalos en *Python* como *DataFrame*.

  - *pandas.read_csv*(direccion_ficheiro,*args): para abri-los ficheiros tipo *csv* e importalos en *Python* como *DataFrame*.

  - *pandas.read_json*(direccion_ficheiro,*args): para abri-los ficheiros tipo *json* e importalos en *Python* como *DataFrame*.

- *pandas.read_table*(direccion_ficheiro,*args): para abri-los ficheiros de case calquera tipo e importalos en *Python* como *DataFrame*. Neste caso cómpre indicarlle cal é o separador de columnas a través dos argumentos da función.

Representación dos datos

Para unha correcta interpretación dos datos, soe ser útil a súa representación gráfica e, máis concretamente, unha representación *axeitada*. Neste apartado veremos como tratar coas representacións en *Python* a través da *librería* por excelencia (a pesar de existiren outras): *matplotlib*. E dentro da mesma, *pyplot ou pylab* (son case equivalentes). Para evitar nomear continuamente á *librería* completa para os exemplos, e dado que se soe importar cun alias, diremos que importamos *matplotlib.pyplot/matplotlib.pylab* como *plt*.

Para traballar cunha visualización rápida de dúas variables x, y, unha dependente da outra, abonda cunha única liña: *plt.plot(x,y)* ou, como moito e para que non se nos solapen varias representacións, engadiríamos antes un *plt.figure()*. Non obstante, cando queremos manipular unha figura e moldeala ó noso gusto, hai que ir un paso máis alá. Dunha figura de *matplotlib* pode modificarse case todo, pero primeiro hai que saber que exista e logo saber *como*. Neste apartado veremos brevemente algunhas nocións básicas sobre as que traballar con estas figuras.

O primeiro é que pasemos a traballar coa figura e coas representacións gráficas dentro dela (o chamados *axes*) como obxectos. Do mesmo xeito que traballamos con *arrays*, cos *DataFrame* con listas ou con calquera outra variable que teña unhas propiedades asociadas, deberiamos da-lo paso de tratar a figura como outro obxecto máis, e para elo é convinte gardala nunha variable nova, á que logo chamar.

$$\left\{\text{figura=plt.figure()}\right.$$

O que se acaba de crear é a figura, pero sen ningunha representación nela. Sen eixos de coordenadas. O seguinte paso será engadilos, e para iso hai tamén diferentes camiños- Por que esta diferenza? Porque, como se amosa da figura 1, cada elemento mencionado ten unha connotación distinta en *Python*. Con *add_subplot(fcp)* engadimos unha única representación na posición p dunha división de f filas e c columnas, onde f, c, p son números enteiros. Con *plt.subplots(f,c)* creamos fxc representacións gráficas (filas x columnas) e maila figura que as contén. E, como se fixo antes, é convinte almacena-la figura (*figure*)e as representacións (*axes*) nunha nova variable, que para o subplot conterá só un obxecto pero para o subplots conterá dous: a figura e máis tódolas representacións, que se atoparán nun *array*. Para o caso dun único subplot:

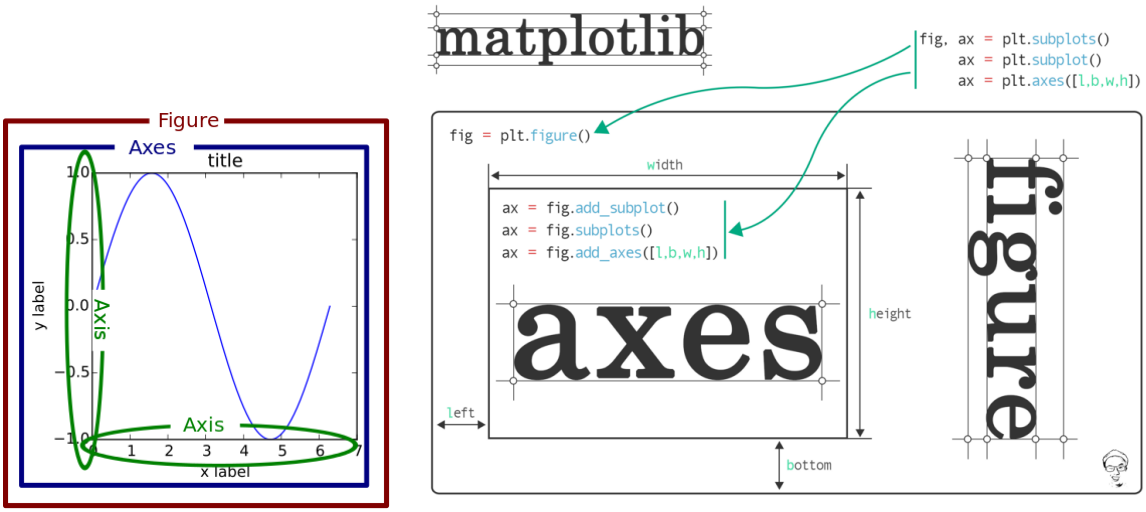

Fig. 1: Dúas exemplificacións da relación figura-eixos

$$\begin{cases} \text{figura=plt.figure()} \\ \text{eixos=figura.add_subplot(111)} \end{cases}$$

Sobre estes eixos creados, podemos xa face-la presentación das variables. *eixos* é un obxecto tipo *axes* de *matplotlib*, e unha das súas funcións é, precisamente, a da representación. Podemos aplicar o *plot* como función do mesmo *eixos*

$$\begin{cases} \text{figura=plt.figure()} \\ \text{eixos=figura.add_subplot(111)} \\ \text{eixos.plot(x,y)} \end{cases}$$

Cal é a vantaxe de telo feito así? Que agora podemos traballar sobre a variable (ou o *array* no caso de termos feito varios) sen ter que crear novas representacións. E qué se pode modificar? Pois, entre outras cousas, o amosado na figura 2:

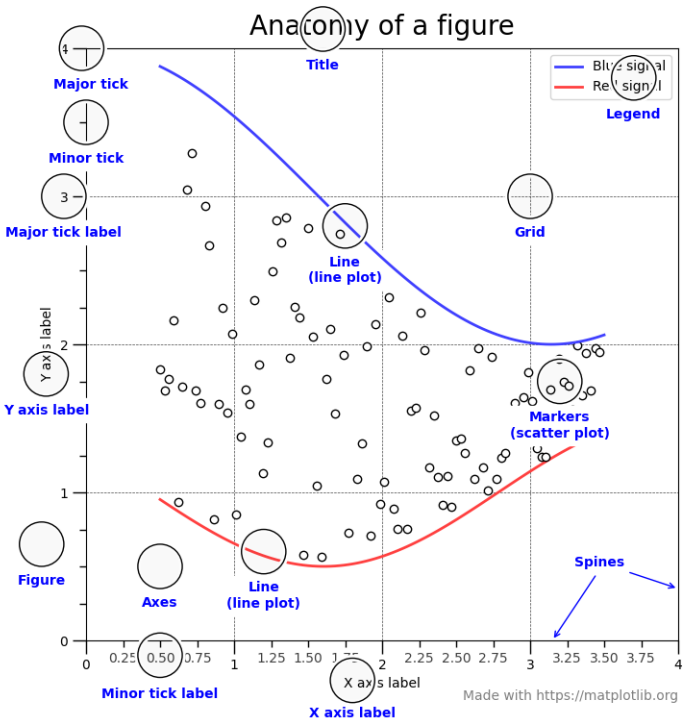

Fig. 2: Anatomía dunha figura en *matplotlib*.

Sobre como e que cambiarlle, como cadaquén ten a súa idea particular de como preparar unha figura, vouvos remitir ás seguintes páxinas, onde poderedes atopar *case* todo o que buscades. Por un lado, a propia páxina da documentación dos eixos creados, que son un obxecto *axes*: `https://matplotlib.org/stable/api/axes_api.html`. Outra parte importante é modificar cousas na figura: `https://matplotlib.org/stable/api/_as_gen/matplotlib.figure.Figure.html#matplotlib.figure.Figure`. E nestas últimas, algúns exemplos de cousas que se poden facer: a do propio *matplotlib* `https://matplotlib.org/3.3.3/api/_as_gen/matplotlib.pyplot.figure.html` e unha externa `https://python-graph-gallery.com/`.

Axustes

Unha vez recollidos os datos, o seguinte paso no tratamento dos mesmos é describi-lo seu comportamento para logo poder predicir sucesos futuros. Estamos a falar, no noso campo, do axuste a un modelo matemático, a unha función. Sen entrar en detalle, adoitamos aplicar neste caso a minimización por

mínimos cadrados, que é unha das técnicas de optimización habituais, pero non a única. Para modelos non lineais, por exemplo, existe o algoritmo de Levenberg-Marquardt, o de Gauss-Newton, o BFGS... moitos deles están xa implementados en diferentes *librerías* de *Python*. Para os posibles problemas aos que nos poidamos enfrontar, propoñemos aquí dúas delas: o axuste a un polinomio con *numpy* e o axuste a unha función previamente definida (por nós ou xa existente) a través do *curve_fit*, da librería *scipy.optimize* (usaremos *so* para abreviar).

- *numpy.polyfit(x, y, deg, full=True/False, w, cov=True/False,*kwargs)*
  x,y: datos das variables x e y, en formato *array*
  deg: grao do polinomio que se busca
  full: (por defecto, False) indica se ao aplica-lo axuste: False: devolve unicamente o valor dos coeficientes do polinomio (e a matriz de covarianza se *cov* está activado). True: devolve outra información do axuste, como os residuos.
  w: (por defecto, None) variable tipo *array*, opcional, que conterá ós pesos estatísticos dos datos, expresados, de se-lo caso, en termos da incerteza e non da varianza.
  cov: (por defecto, False) se está activada, devovle matriz de covarianza
  Para máis información: `https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html`

- *so.curve_fit(funcion, x, y, p0, sigma, absolute_sigma, bounds, method, *kwargs)*
  funcion: función tipo *Python* que queremos axustar.
  x,y: datos das variables x e y, en formato *array*
  p0: (por defecto, None) semente (valores iniciais suxeridos) para os coeficientes de axuste
  sigma: (por defecto, None). Un *array* que contén as incertezas dos nosos datos e que se é de máis dunha dimensión será interpretado como a covarianza.
  absolute_sigma: (por defecto, False) para que a **sigma** sexa interpretada correctamente cos seus valores absolutos e non relativos, debe ser posto como True.
  bounds: (por defecto -inf, +inf) límites inferior e superior sobre os que busca-los valores dos parámetros axustables. Non funciona co algoritmo por defecto (*lm*)??
  method: (por defecto "lm') tipo de algoritmo empregado para a minimización.
  Para máis información: `https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html`

Exportación

Cando xa rematamos coa análise dos datos e o seu tratamento, pode que nos interese exporta-los resultados a un documento externo a Python. Poderiamos, de novo, volver a gardalo nun *.txt, .dat, .xls/.xlsx, .csv, .ods...* segundo as nosas preferencias ou necesidades. Do mesmo xeito que coa importanción, a función *open()* de *Python*, co argumento para escribir, permitiríanos abrir un documento baleiro e incorporarlle toda a información que queriamos, mais deberiamos darlle un formato previo (introducir as tabulacións, os expazos, os saltos de liña...). Como xa fixemos unha breve presentación con *Pandas*, imos botar man desta mesma librería para a exportación xa que, como se indicou, está especificamente deseñada para traballar con datos, e isto inclúe tamén ás exportacións. *Pandas* permite a exportación de datos nos mesmos formatos que permite a súa lectura e, ademais, permite a exportación dun *DataFrame* como táboa de LaTeX. Veremos este caso e unha das exportacións a un formato, xa que o resto son equivalentes.

Supoñamos que temos un *DataFrame* xa creado e listo para importar. Chamémoslle *df*:

- *df.to_latex(buf=None, columns=None, col_space=None, header=True, index=True, *args)* é unha das funcións vinculadas ao propio *DataFrame* que permite exportalo como táboa de LaTeX. Os valores indicados entre parénteses son os que aparecen por defecto e que vos poden resultar máis útiles. Como tales, non cómpre darlles ningún valor se o que está por defecto cumpre co que buscades. Aquí aparecen algunhas das posibilidades e explicacións:
  buf: dirección (co nome e extensión do ficheiro) onde imos almacena-la táboa. En caso de mantelo como *None*, a táboa sairá por pantalla xa formateada.
  columns: (por defecto, None e como entradas válidas recolle unha lista ou un marcador de columna) Unha selección das columnas que se queira representar (por se non querémo-la táboa enteira, que sería o valor por defecto).
  col_space: (por defecto, None e como entradas válidas admite enteiros) Ancho mínimo de cada columna.
  header: (por defecto, True e como entradas válidas admite True/False, unha lista ou cadeas de caracteres) Escribe os nomes das columnas. Se se marca *True*, dálle os nomes que aparecen no *DataFrame*.
  index: (por defecto, True e como entradas válidas admite True/False) Escribe os nomes das filas ou os seus índices no caso de que non teñan marcadores asignados. Se se marca *True*, dálle os nomes que aparecen no *DataFrame*.

- *df.to_excel(direccion, sheet_name='Sheet1',columns=None, header=True, index=True, *args)* é unha das funcións vinculadas ao propio *DataFrame* que permite exportalo como táboa de *Excel*. Os valores indicados entre parénteses son os que aparecen por defecto e que vos poden resultar máis útiles. Como tales, non cómpre darlles ningún valor se o que está por defecto cumpre co que buscades. Aquí aparecen algunhas das posibilidades e explicacións:
  direccion: dirección e nome do ficheiro excel que queremos crear, incluída a extensión! Este argumento é obrigatorio pois, ó contrario que coa táboa previa, o formato non vai saír de forma correcta por pantalla. Se só se lle indica o nome do ficheiro, gardarao no mesmo lugar onde executámo-lo script de *Python*.
  sheet_name: (por defecto, 'Sheet1' e como entradas válidas admite cadeas de caracteres) Nome da páxina de *Excel* onde gardarémo-los datos.
  columns: (por defecto, None e como entradas válidas recolle unha lista ou un marcador de columna) Unha selección das columnas que se queira representar (por se non querémo-la táboa enteira, que sería o valor por defecto).
  header: (por defecto, True e como entradas válidas admite True/False, unha lista ou cadeas de caracteres) Escribe os nomes das columnas. Se se marca *True*, dálle os nomes que aparecen no *DataFrame*.
  index: (por defecto, True e como entradas válidas admite True/False) Escribe os nomes das filas ou os seus índices no caso de que non teñan marcadores asignados. Se se marca *True*, dálle os nomes que aparecen no *DataFrame*.

EXERCICIO 1: Importa-lo ficheiro *casos_ tecnica_ ccaa.csv* e realiza-la representación do número de casos dunha comunidade autónoma (tamén se acepta os dunha provincia ou os casos totais) fronte ás datas. Pode escollerse outra das variables que non sexa número de casos. Hai que ter coidado coas datas, debemos traballalas como *numpy.array.astype("datetime64")* ou *pandas.to_ datetime()*, formato aaaa-mm-dd.

EXERCICIO 2: Importa-lo ficheiro *casos_ tecnica_ ccaa.csv*, realiza-la representación do número de casos dunha comunidade autónoma (tamén se acepta os dunha provincia ou os casos totais) e, para as datas comprendidas entre o 01-03-2020 e 01-07-2020, facer un axuste dos datos a unha función *log-normal*:

$$y = \frac{A}{(x\sigma\sqrt{2\pi})} e^{\frac{-(\log x - \mu)^2}{2\sigma^2}} \tag{1}$$

EXERCICIO 3: Importa-lo ficheiro *casos_ tecnica_ ccaa.csv* e realiza-la representación do mesmo exemplo que os exercicios anteriores e outros tres casos máis (outras variables, outras comunidades autónomas, provincias, estatais, datos normalizados, etc.), na mesma figura pero en representacións separadas.

EXERCICIO 4: Importa-lo ficheiro *casos_ tecnica_ ccaa.csv*, realiza-la representación do mesmo exemplo que os exercicios 1 e 2 e, para as datas comprendidas entre o 01-03-2020 e 01-07-2020 facer un axuste dos datos a unha función *log-normal* e determina-los puntos de corte (puntos críticos) nos cales se superou o límite de casos por cada 100000 habitantes para o total da serie de datos.